# Pothole Detection with YOLO — Training & Evaluation Guide

Nov 2, 2026 · @Judie

## Overview

A single-class pothole detector (YOLO, Ultralytics) reaches 0.934 mAP50 on validation and 0.938 on test after training on a Kaggle dataset in Google Colab.

| Item | Value |
|---|---|
| Task | Object detection, 1 class ( `pothole` ) |
| Dataset | Pothole Detection Dataset: YOLOv11 Optimized (CC BY 4.0) |
| Split sizes | 3,345 train / 397 val / 198 test images |
| Model | `yolo26n.pt` , 2.39 M parameters, 5.4 GFLOPs |
| Hardware | Colab Tesla T4 GPU |

| Split | Precision | Recall | mAP50 | mAP50-95 |
|---|---|---|---|---|
| Validation (973 boxes) | 0.895 | 0.867 | 0.934 | 0.668 |
| Test (433 boxes) | 0.913 | 0.872 | 0.938 | 0.676 |

Inference takes about 5 ms per image on the T4.

## 1. Setup and dataset download

Switch Colab to a T4 GPU (Runtime > Change runtime type), then install the libraries and confirm the GPU is visible.

```
%pip install -q -U ultralytics kaggle opencv-python-headless

import torch, ultralytics
print("Ultralytics:", ultralytics.__version__)
print("GPU available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
```

Mount Google Drive (optional, keeps results after the session ends) and authenticate with Kaggle. Paste the API token when prompted, or store it as a Colab Secret named `KAGGLE_API_TOKEN`.

```
from google.colab import drive
drive.mount('/content/drive')

import os
from getpass import getpass
os.environ["KAGGLE_API_TOKEN"] = getpass("Paste your Kaggle API token: ")
```

Download and unzip the dataset.

```
kaggle datasets list -s pothole
mkdir -p /content/dataset
kaggle datasets download \
    -d muskanverma24/pothole-detection-dataset-yolov11-optimized \
    -p /content/dataset --unzip
```

## 2. Dataset checks and YAML

All 3,345 + 397 + 198 images have matching label files, there are 10,111 pothole boxes in total, and no label line is malformed. The check below reproduces this.

```
from pathlib import Path
from collections import Counter

base = Path("/content/dataset")
for split in ["train", "valid", "test"]:
    n_img = len(list((base / split / "images").glob("*")))
    n_lbl = len(list((base / split / "labels").glob("*.txt")))
    print(split, "images:", n_img, "labels:", n_lbl)

class_counts, invalid = Counter(), []
for lf in base.glob("*/labels/*.txt"):
    for i, line in enumerate(lf.read_text().splitlines(), 1):
        parts = line.split()
        if not parts:
            continue
        if len(parts) != 5:
            invalid.append((str(lf), i, line)); continue
        class_counts[int(parts[0])] += 1
print(class_counts, "malformed:", len(invalid))
```

The YAML shipped with the dataset uses relative paths ( `../train/images` ) that fail in Colab. Write a corrected one with an absolute `path` .

```
import yaml
config = yaml.safe_load(open("/content/dataset/data.yaml"))
fixed = {
    "path": "/content/dataset",
    "train": "train/images",
    "val": "valid/images",
    "test": "test/images",
    "nc": config["nc"],
    "names": config["names"],
}
yaml.safe_dump(fixed, open("/content/pothole_colab.yaml", "w"),
sort_keys=False)
```

## 3. Training

Training fine-tunes `yolo26n.pt` for up to 50 epochs and stops early after 10 epochs without improvement.

```
from ultralytics import YOLO

model = YOLO("yolo26n.pt")
results = model.train(
    data="/content/pothole_colab.yaml",
    epochs=50,
    imgsz=640,
    batch=-1,
    patience=10,
    workers=2,
    project="/content/runs/pothole",
    name="yolo26n",
    seed=42,
    plots=True,
)
RUN_DIR = model.trainer.save_dir   # actual run folder (yolo26n, yolo26n-2,
...)
BEST = RUN_DIR / "weights" / "best.pt"
```

| Parameter | Value | Purpose |
|---|---|---|
| Initial weights | `yolo26n.pt` | Starting model |
| epochs | 50 | Maximum training epochs |
| imgsz | 640 | Input image size (px) |
| batch | -1 | Auto batch size |
| patience | 10 | Early-stopping patience (epochs) |
| workers | 2 | Data-loading workers |
| seed | 42 | Reproducibility |
| plots | True | Save training curves |

The run folder holds `weights/best.pt`, `weights/last.pt`, `results.csv` and `results.png`. Always read the folder from `model.trainer.save_dir`: a re-run creates `yolo26n-2`, `yolo26n-3` and so on.

# 4. Evaluation, prediction and export

Evaluate the best weights on both held-out splits; results match the table in the overview.

```
best = YOLO(str(BEST))
val_metrics  = best.val(data="/content/pothole_colab.yaml", split="val",
imgsz=640)
test_metrics = best.val(data="/content/pothole_colab.yaml", split="test",
imgsz=640)
print(test_metrics.box.mp, test_metrics.box.mr, test_metrics.box.map50,
test_metrics.box.map)
```

Run predictions on test images (confidence threshold 0.25) and save the annotated output.

```
import random
from pathlib import Path
imgs = sorted(Path("/content/dataset/test/images").glob("*.jpg"))
sample = random.Random(42).sample(imgs, 6)
best.predict(source=[str(p) for p in sample], imgsz=640, conf=0.25,
save=True)
```

Download the weights to keep them.

```
from google.colab import files
files.download(str(BEST))
```

## Video pipeline (optional)

`adas_live.py` loads `best.pt` from the working directory. The earlier run failed with `FileNotFoundError: best.pt` because the weights had not been copied there. Copy them first, then run the script on a road video.

```
import shutil
shutil.copy(BEST, "best.pt")
!python adas_live.py road_video.mp4
```